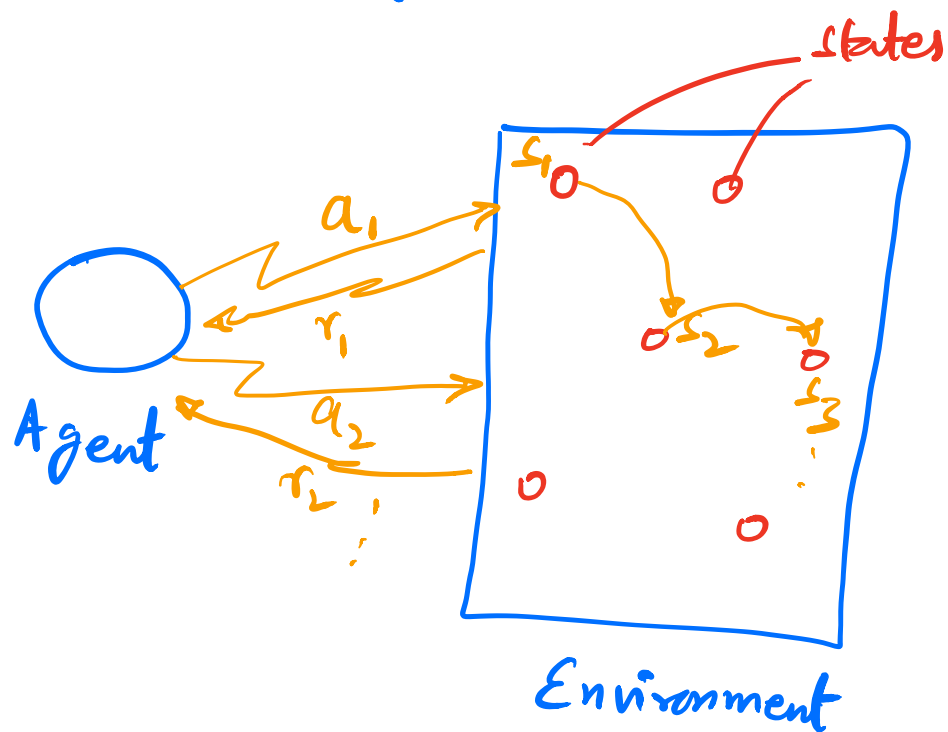


Reinforcement Learning

RL: Learning based on interactions with environment



Goal: Maximize long-term reward

immediate rewards : $r_1, r_2, \dots$

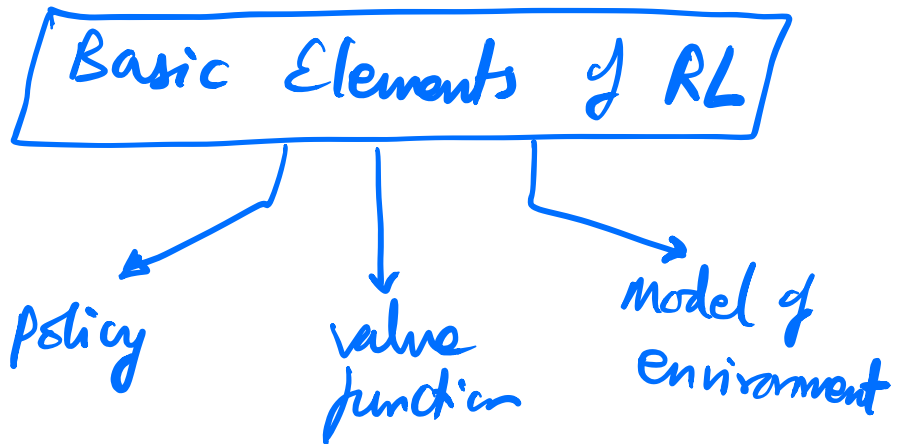
Key Ingredients to an RL Problem

(i) state of the system

[summarizes the key characteristics of the environment]

(ii) Action: Control picked by the agent or decision maker

(iii) Single-stage Reward: Numerical reward obtained from environment for the actions chosen by agent



Policy: A map from the states of environment to actions

- Can be in form of a **look up table**

Example: states = $\{1, 2, 3, 4, 5\}$

Actions = $\{a, b\}$

States	Actions
1	b
2	a
3	b
4	b
5	a

Such policies are also called deterministic policies

- Stochastic policies

Example:

$$\begin{array}{cc} \text{states} & \\ \swarrow & \\ p(b|1) = 0.9, & p(a|1) = 0.1 \\ p(b|2) = 0.3, & p(a|2) = 0.7 \\ \downarrow & \downarrow \\ \text{action} & \end{array}$$

- Reward that we obtain in a given state when an action is chosen is random in general

Example

Consider a system with states 1, 2 and 2 actions — a, b.

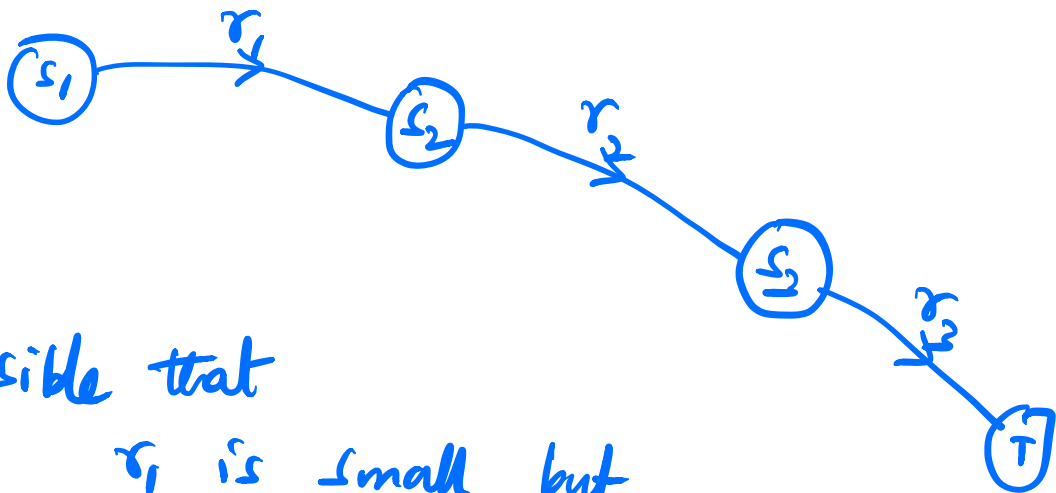
Suppose there are 3 possible rewards R_1, R_2, R_3 .

States	Actions	Rewards
1	a	R_1 w.p. 0.6 R_2 w.p. 0.4
1	b	R_2 w.p. 0.6 R_3 w.p. 0.4
2	a	R_1 w.p. 0.7 R_2 w.p. 0.1 R_3 w.p. 0.2
2	b	R_3 w.p. 1

Value Function:

- A function from states to real numbers
- Help identify a good course of actions in the long run
- Specify the long-term desirability of a state

Example



Possible that

r_1 is small but

$V(s_1)$ is high

because σ_2 & σ_3 are large.

- Decisions made by an RL agent are based on estimates of value function & not immediate rewards obtained.

Model of the Environment

- Mimics the behavior of the environment
- Can be used for predicting next states and rewards given the current state and action

Time line



Model based RL methods:

Make use of models for planning

Model free RL methods:

Do not involve planning but are more trial and error based approaches.

The real learning happens with model free RL methods

Exploration vs. Exploitation

Exploitation: selecting actions tried by agent that it has discovered yield a high reward.

Exploration: selecting actions never tried before to see what rewards they might give

Note: Exploitation & Exploration are conflicting objectives, hence the tradeoff.

Example: Tic-Tac-Toe

X	X	O
	O	

Player 1: X

Player 2: O

Suppose: Player of interest
 - Player 1
 (CAL agent)

Assume Player 2 plays as
per some policy

• state = $(s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10})$

$s_1, s_2, \dots, s_9 \in \{X, O, e\}$

$s_{10} \in \{X, O\}$

[Who started the game]

Suppose

$$s = (x, x, 0, 0, x, e, 0, e, e | x)$$

↓
Player
1
started
the
game

• Action for player 1:

Which of the e-entries to change
to x

$$s_t = (x, x, 0, 0, x, e, 0, \text{e}, e | x)$$

x	x	0
0	x	
0	x	

$$s_{t+1} = (x, x, 0, 0, x, e, 0, x, e | x)$$

Reward:

$$r_t = \begin{cases} 1 & \text{if game ends at time } t \text{ with win} \\ 0 & \text{o.w} \end{cases}$$

value: Estimate prob. of a win from that state.

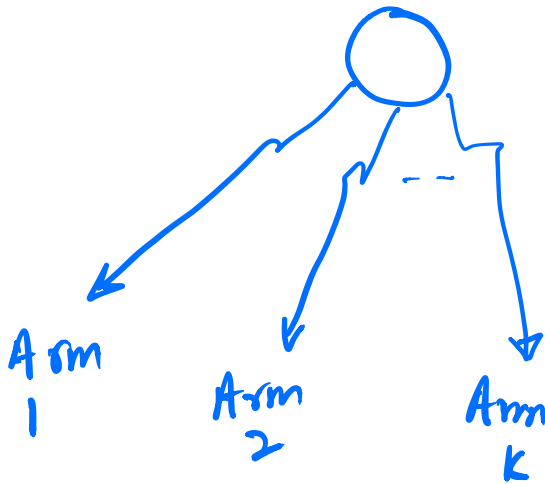
Book: R. S. Sutton and A. Barto,
Reinforcement Learning, 2018
(2nd ed), MIT Press

→ Rich Sutton

Book page → download PDF

Chapter 1 — Over.

Chapter 2: Multi-armed Bandits



- Assume reward obtained on each pull of the arm is random
- Reward from each arm comes from some dist. that is not known

Goal: Maximize the expected total reward

Let $q_*(a) \triangleq E[R_t | A_t = a]$

Action-value Methods

- Estimate the values of actions
- use the estimates obtained for action selection decisions

Let $Q_t(a) = \frac{\text{sum of rewards obtained when action } a \text{ is taken by time } t}{\text{no. of times action } a \text{ is taken by } t}$

$$= \frac{\sum_{i=1}^t R_i I_{\{A_i = a\}}}{\sum_{i=1}^t I_{\{A_i = a\}}}$$

Here $I_{\{A_i = a\}} = \begin{cases} 1 & \text{if } A_i = a \\ 0 & \text{o.w.} \end{cases}$

Let $A_t \triangleq \underset{a}{\operatorname{argmax}} Q_t(a)$

↓
Greedy action selection

- Greedy action selection exploits current knowledge to maximize reward

- Does not sample actions not tried previously.

Alternative: Be greedy most of the time but once in a while explore other actions also.

ϵ -greedy policy:

$$A_t = \begin{cases} \arg\max_a Q_t(a) & \text{w.p. } 1-\epsilon \\ \text{random action} & \text{w.p. } \epsilon \end{cases}$$

Here $0 < \epsilon < 1$ is given

[Many times, $\epsilon = 0.1, 0.2$]

Advantage of ϵ -greedy: In the limit
as $t \rightarrow \infty$, all actions are sampled an
infinite no. of times, thereby ensuring that
 $Q_t(a) \rightarrow Q_*(a) \quad \forall a$

Suppose $Q_n = \frac{1}{n} [R_1 + R_2 + \dots + R_n]$

Assume for simplicity we only pick
action a .

$$\begin{aligned} Q_{n+1} &= \frac{1}{n+1} \sum_{i=1}^{n+1} R_i \\ &= \frac{1}{n+1} \left(\sum_{i=1}^n R_i + R_{n+1} \right) \end{aligned}$$

$$= \frac{n}{n+1} \cdot \underbrace{\frac{1}{n} \sum_{i=1}^n R_i}_{Q_n} + \frac{1}{n+1} R_{n+1}$$

$$= \frac{n Q_n}{n+1} + \frac{R_{n+1}}{n+1}$$

$$= Q_n + \frac{1}{n+1} [R_{n+1} - Q_n]$$

$$\left(\text{Since } \frac{n}{n+1} Q_n \right.$$

$$= Q_n - \frac{1}{n+1} Q_n \Big)$$

Algorithm:

$$Q_{n+1} = Q_n + \frac{1}{n+1} (R_{n+1} - Q_n)$$

Structure:

New Estimate $\leftarrow$ Old Estimate

+ stepsize $\left[\underbrace{\text{target} - \text{old estimate}}_{\text{Error in estimate}} \right]$

Error in estimate

A Simple bandit Algorithm:

Initialize: for $a = 1, \dots, k,$

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{w.p. } 1-\epsilon \\ \text{random action} & \text{w.p. } \epsilon \end{cases}$

$$R \leftarrow \text{bandit}(A)$$

↳ Takes action A

& returns a reward

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \left(\frac{1}{N(A)} \right) [R - Q(A)]$$

↓ step-size

$$Q = [Q(1), Q(2), \dots, Q(k)]$$

- So far we have seen the case where the dynamics of the RL problem is stationary.

Q) What do we do if the dynamic is non-stationary?

Consider a bandit problem where dynamic is nonstationary.

- Need to give more weight to recent rewards as opposed to rewards received in past.
- Constant step size helps here

Consider the modified update rule for θ_n :

$$Q_{n+1} = Q_n + \alpha [R_n - Q_n]$$

where $\alpha \in (0,1)$ is a constant

$$= \alpha R_n + (1-\alpha)Q_n$$

$$= \alpha R_n + (1-\alpha)(\alpha R_{n-1} + (1-\alpha)Q_{n-1})$$

$$= \alpha R_n + (1-\alpha)\alpha R_{n-1}$$

$$+ (1-\alpha)^2 (\alpha R_{n-2} + (1-\alpha)Q_{n-2})$$

$$= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 \alpha R_{n-2}$$

$$+ (1-\alpha)^3 Q_{n-2}$$

⋮

Finally,

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^n \alpha (1-\alpha)^{n-i} R_i$$

• Q_{n+1} is a weighted average of

$Q_1, R_1, R_2, \dots, R_n$

• R_n gets a weight α

$R_{n-1} \quad \dots \quad \alpha(1-\alpha)$

$\vdots$
 $R_1 \quad \dots \quad \alpha(1-\alpha)^{n-1}$

$Q_1 \quad \dots \quad (1-\alpha)^n$

Fading memory on past rewards.

Sum of weights

$$(1-\alpha)^n + \sum_{i=1}^n \alpha (1-\alpha)^{n-i}$$

$$= (1-\alpha)^n + \alpha \left(1 + (1-\alpha) + (1-\alpha)^2 + \dots + (1-\alpha)^{n-1} \right)$$

$$= (1-\alpha)^n + \alpha \frac{1 - (1-\alpha)^n}{1 - (1-\alpha)}$$

$$= 1$$

Remark: In stationary problems,
it makes better sense to have

diminishing step size (of the form $\frac{1}{N(A)}$) while in nonstationary problems, it makes better sense to have constant step-size for better tracking ability.

2nd Action Selection Strategy

Upper Confidence Bound (UCB) based action selection.

Action Selection rule:

Exploitation
term

Exploration
term

$$A_t = \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Here $N_t(a)$ = no. of times action a is selected before time t .

$c > 0$: Controls the degree of exploration

Note

- When $N_t(a) = 0$, a is the maximizing action & a must be selected (since 2nd term $= \infty$).
- When $N_t(a)$ is large, for all a ,

Since $\ln t$ is small (note:
 $\ln 10^6 = 6$), the actions are
decided mainly by the $Q_t(a)$ values.

- Action selection probabilities may
be chosen in general according to
Gibbs or Boltzmann distributions

$$P(A_t = a) = \frac{e^{H_t(a)}}{\sum_{b=1}^K e^{H_t(b)}} \triangleq \pi_t(a)$$

Here $H_t(a)$: preference function

Example: Initially all action preferences may be chosen the same.

Suppose $H_0(a) = 0 \forall a$

Then $\pi_0(a) = \frac{1}{K} \forall a$